

License

- **Open Source:** Apache License 2.0

For full details, please see the [LICENSE](#) file in the repository.

Go

```
go get github.com/mikhaildadaev/uuid
```

Последняя стабильная версия `uuid` — **v1.26.11**.

Run Test

```
go test ./...  
go test -bench=. ./...  
go test -cover ./...  
go test -race ./...
```

Key Features

- **UUIDv1..v8 и Null** — Генерация всех версий UUID и SQL-совместимого null UUID;
- **Парсинг и Валидация** — Парсинг и проверка стандартных и null UUID строк;
- **Извлечение Метаданных** — Извлечение временной метки, последовательности, узла, POSIX, варианта и версии;
- **Сериализация** — Полная поддержка Binary, JSON и Text marshal/unmarshal;
- **SQL Интеграция** — Нативная поддержка database/sql driver с интерфейсами Scan/Value;
- **Zero-Allocation** — Быстрые, без выделений памяти, основные процедуры на всех критических путях.

Limits

- **V2:** `posix` должен быть `0..255` ;

- **V3/V5:** `name` должен быть `1..512` символов длиनावбайтах;
- **V8:** `node` должен быть `0..16383` ;

Строки `name` длиной 0 байт или более 512 байт возвращают null UUID для предотвращения выделения памяти.

API / Core / Конструкторы

На этой странице описано создание каждой версии UUID: от совместимого с SQL `NULL` до основанной на метке времени V7 и пользовательской V8. Начните с **NewV4** для общего использования или **NewV7** для сортируемых идентификаторов.

NewNull

Создаёт UUID, совместимый с SQL-значением `NULL` .

```
un := uuid.NewNull()
fmt.Println(un.IsZero())
fmt.Println(un.String())
```

Output TEXT:

```
true
00000000-0000-0000-0000-000000000000
```

NewV1

Создаёт UUID версии 1 на основе текущей метки времени и MAC-адреса локальной машины илислучайногоузла, если *MAC* недоступен.

```
u1 := uuid.NewV1()
fmt.Println(u1.Version())
```

Output TEXT:

1

NewV2

Создаёт UUID версии 2 *DCESecurity* с использованием текущей метки времени, MAC-адреса локальной машины и POSIX UID/GID.

```
u2 := uuid.NewV2(posix)
fmt.Println(u2.Version())
```

Output TEXT:

2

NewV3

Создаёт UUID версии 3 путём хеширования идентификатора пространства имён и имени с помощью MD5.

```
u3 := uuid.NewV3(uuid.NameSpaceDNS, name)
fmt.Println(u3.Version())
```

Output TEXT:

3

NewV4

Создаёт UUID версии 4 с использованием криптографически безопасных случайных чисел.

```
u4 := uuid.NewV4()  
fmt.Println(u4.Version())
```

Output TEXT:

```
4
```

NewV5

Создаёт UUID версии 5 путём хеширования идентификатора пространства имён и имени с помощью SHA-1.

```
u5 := uuid.NewV5(uuid.NamespaceDNS, name)  
fmt.Println(u5.Version())
```

Output TEXT:

```
5
```

NewV6

Создаёт UUID версии 6 полевая совместимость с *UUIDv1* на основе текущей метки времени и MAC-адреса локальной машины или случайного узла, если *MAC* недоступен.

```
u6 := uuid.NewV6()  
fmt.Println(u6.Version())
```

Output TEXT:

```
6
```

NewV7

Создаёт UUID версии 7 на основе метки времени, лексикографически сортируемый с использованием текущей метки времени Unix в миллисекундах и случайных битов.

```
u7 := uuid.NewV7()  
fmt.Println(u7.Version())
```

Output TEXT:

```
7
```

NewV8

Создаёт UUID версии 8 пользовательский, зависящий от поставщика с использованием идентификатора узла и текущей метки времени.

```
u8 := uuid.NewV8(node)  
fmt.Println(u8.Version())
```

Output TEXT:

```
8
```

API / Core / Методы

На этой странице описаны все методы экземпляров UUID и NullUUID — от базовых операций, таких как **String** и **Version**, до расширенного извлечения метаданных, таких как **Timestamp**, **Node** и **Sequence**. Каждый метод работает со всеми версиями UUID, от V1 до V8.

NULLUUID

IsZero

Возвращает `true`, если NullUUID недействителен *SQLNULL* или базовый UUID является нулевым значением.

```
var nu uuid.NullUUID
fmt.Println(nu.IsZero())
nu.Scan("01968727-8c7e-8000-87cb-bdba4f634d9f")
fmt.Println(nu.IsZero())
```

Output TEXT:

```
true
false
```

String

Возвращает каноническую текстовую форму 8 — 4 — 4 — 4 — 12 для валидного UUID или `00000000-0000-0000-0000-000000000000` для null-значения.

```
var nu uuid.NullUUID
fmt.Println(nu.String())
nu.Scan("01968727-8c7e-8000-87cb-bdba4f634d9f")
fmt.Println(nu.String())
```

Output TEXT:

```
00000000-0000-0000-0000-000000000000
01968727-8c7e-8000-87cb-bdba4f634d9f
```

Validate

Проверяет NullUUID. Возвращает `nil` как для null, так и для валидных UUID *null* считается валидным в контексте *SQL*. Возвращает ошибку только если UUID присутствует, но имеет неверный формат.

```
var nu uuid.NullUUID
fmt.Println(nu.Validate())
nu.Scan("01968727-8c7e-8000-87cb-bdba4f634d9f")
fmt.Println(nu.Validate())
```

Output TEXT:

```
<nil>
<nil>
```

UUID

Bytes

Возвращает UUID в виде 16-байтового среза.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
fmt.Printf("%x\n", uu.Bytes())
```

Output TEXT:

```
019687278c7e800087cbbdba4f634d9f
```

Equal

Сравнивает два UUID на равенство. Возвращает true, если оба UUID представляют одно и то же значение.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
other := uu
fmt.Println(uu.Equal(other))
```

Output TEXT:

```
true
```

Info

Возвращает удобочитаемую сводку UUID: версию, вариант, метку времени, последовательность, узел и POSIX.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
fmt.Println(uu.Info())
```

Output TEXT:

```
UUID: 01968727-8c7e-8000-87cb-bdba4f634d9f
VAR.: RFC4122
VER.: 8
FORM: TTTTTTTT-TTTT-8SSS-VNNN-RRRRRRRRRRRR
INFO: TIME (1-milliseconds interval since 1970-01-01) + SEQUENCE (0-4095) + NOI
TIME: 1746024238206 [2025-04-30 14:43:58.206 +0000 UTC]
SEQ.: 0
NODE: 1995
RAND: bdba4f634d9f
```

IsZero

Возвращает true, если UUID является нулевым значением все16байтравнынулю.

```
uu := uuid.NewNull()  
fmt.Println(uu.IsZero())
```

Output TEXT:

```
true
```

Node

Возвращает идентификатор узла для UUID версий V1, V2, V6 и V8. Возвращает 0 для других версий.

```
uuidNode := 1995  
u8 := uuid.NewV8(uuidNode)  
fmt.Println(u8.Node())
```

Output TEXT:

```
1995
```

Posix

Возвращает POSIX UID/GID для UUID версии V2. Возвращает 0 для других версий.

```
uuidPosType := 0  
uuidPosValue := 501  
u2 := uuid.NewV2(uuidPosType, uuidPosValue)  
fmt.Println(u2.Posix())
```

Output TEXT:

```
UID 501
```

Sequence

Возвращает поле тактовой последовательности для UUID версий V1, V2, V6 и V7.
Возвращает 0 для других версий.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"  
uu, err := uuid.Parse(uuidV8String)  
if err != nil {  
    fmt.Println(err)  
}  
fmt.Println(uu.Sequence())
```

Output TEXT:

```
0
```

String

Возвращает UUID в канонической текстовой форме: 8-4-4-4-12 шестнадцатеричных цифр с дефисами.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"  
uu, err := uuid.Parse(uuidV8String)  
if err != nil {  
    fmt.Println(err)  
}  
fmt.Println(uu.String())
```

Output TEXT:

```
01968727-8c7e-8000-87cb-bdba4f634d9f
```

Timestamp

Возвращает метку времени, встроенную в UUID версий V1, V2, V6 и V7. Возвращает нулевое время для других версий.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
fmt.Println(uu.Timestamp())
```

Output TEXT:

```
1746024238206
```

Validate

Проверяет UUID и возвращает версию UUID при успехе или ошибку при неудаче.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
fmt.Println(uu.Validate())
```

Output TEXT:

```
<nil>
```

Variant

Возвращает номер варианта UUID. Действительные UUID возвращают 1 стандарт *RFC4122*.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
fmt.Println(uu.Variant())
```

Output TEXT:

```
1
```

Version

Возвращает номер версии UUID от 1 до 8. Возвращает 0 для null UUID.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
fmt.Println(uu.Version())
```

Output TEXT:

```
8
```

API / Marshal / Методы

На этой странице описана сериализация и десериализация UUID в форматы **Binary**, **Json** и **Text**. Каждый метод работает со всеми версиями UUID — от V1 до V8, включая Null.

NULLUUID

MarshalBinary

Кодирует NullUUID в 16-байтовое бинарное представление. Возвращает все нули для null-значения.

```

var nu uuid.NullUUID
data, err := nu.MarshalBinary()
if err != nil {
    fmt.Println(err)
}
fmt.Printf("%x\n", data)
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
nu.Scan(uuidV8String)
data, err = nu.MarshalBinary()
if err != nil {
    fmt.Println(err)
}
fmt.Printf("%x\n", data)

```

Output TEXT:

```

019687278c7e800087cbbdba4f634d9f

```

MarshalJson

Кодирует NullUUID в строку Json. Возвращает null для null-значения.

```

var nu uuid.NullUUID
data, err := nu.MarshalJson()
if err != nil {
    fmt.Println(err)
}
fmt.Println(string(data))
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
nu.Scan(uuidV8String)
data, err := nu.MarshalJson()
if err != nil {
    fmt.Println(err)
}
fmt.Println(string(data))

```

Output TEXT:

```

null
"01968727-8c7e-8000-87cb-bdba4f634d9f"

```

MarshalText

Кодирует NullUUID в каноническую текстовую форму. Возвращает пустую строку для null-значения.

```
var nu uuid.NullUUID
data, err := nu.MarshalText()
if err != nil {
    fmt.Println(err)
}
fmt.Println(string(data))
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
nu.Scan(uuidV8String)
data, err := nu.MarshalText()
if err != nil {
    fmt.Println(err)
}
fmt.Println(string(data))
```

Output TEXT:

```
01968727-8c7e-8000-87cb-bdba4f634d9f
```

UnmarshalBinary

Декодирует NullUUID из 16-байтового бинарного представления. Устанавливает Valid в зависимости от входных данных.

```
var nu uuid.NullUUID
data := []byte{}
if err := nu.UnmarshalBinary(data); err != nil {
    fmt.Println(err)
}
fmt.Println("Valid:", nu.Valid)
fmt.Println("UUID:", nu.UUID)
uuidV8Binary := []byte{0x01, 0x96, 0x87, 0x27, 0x8c, 0x7e, 0x80, 0x00, 0x87, 0x01, 0xb4, 0x34, 0xd9, 0xf0, 0x00, 0x00}
err := nu.UnmarshalBinary(uuidV8Binary)
if err != nil {
    fmt.Println(err)
}
fmt.Println("Valid:", nu.Valid)
fmt.Println("UUID:", nu.UUID)
```

Output TEXT:

```
Valid: false
UUID: 00000000-0000-0000-0000-000000000000
Valid: true
UUID: 01968727-8c7e-8000-87cb-bdba4f634d9f
```

UnmarshalJson

Декодирует NullUUID из Json-строки. Принимает null для SQL NULL или валидную строку UUID.

```
var nu uuid.NullUUID
data := []byte(`null`)
if err := nu.UnmarshalJson(data); err != nil {
    fmt.Println(err)
}
fmt.Println("Valid:", nu.Valid)
fmt.Println("UUID:", nu.UUID)
uuidV8Json := []byte(`"01968727-8c7e-8000-87cb-bdba4f634d9f"`)
err := nu.UnmarshalJson(uuidV8Json)
if err != nil {
    fmt.Println(err)
}
fmt.Println("Valid:", nu.Valid)
fmt.Println("UUID:", nu.UUID)
```

Output TEXT:

```
Valid: false
UUID: 00000000-0000-0000-0000-000000000000
Valid: true
UUID: 01968727-8c7e-8000-87cb-bdba4f634d9f
```

UnmarshalText

Декодирует NullUUID из канонической текстовой формы. Принимает пустую строку для SQL NULL.

```

var nu uuid.NullUUID
data := []byte("")
if err := nu.UnmarshalText(data); err != nil {
    fmt.Println(err)
}
fmt.Println("Valid:", nu.Valid)
fmt.Println("UUID:", nu.UUID)
uuidV8Text := []byte("01968727-8c7e-8000-87cb-bdba4f634d9f")
err := nu.UnmarshalText(uuidV8Text)
if err != nil {
    fmt.Println(err)
}
fmt.Println("Valid:", nu.Valid)
fmt.Println("UUID:", nu.UUID)

```

Output TEXT:

```

Valid: false
UUID: 00000000-0000-0000-0000-000000000000
Valid: true
UUID: 01968727-8c7e-8000-87cb-bdba4f634d9f

```

UUID

MarshalBinary

Кодирует UUID в 16-байтовое бинарное представление.

```

uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
data, err := uu.MarshalBinary()
if err != nil {
    fmt.Println(err)
}
fmt.Printf("%x\n", data)

```

Output TEXT:

```
019687278c7e800087cbbdba4f634d9f
```

MarshalJson

Кодирует UUID в строку Json в стандартном формате 8-4-4-4-12.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
data, err := uu.MarshalJson()
if err != nil {
    fmt.Println(err)
}
fmt.Println(string(data))
```

Output TEXT:

```
"01968727-8c7e-8000-87cb-bdba4f634d9f"
```

MarshalText

Кодирует UUID в каноническую текстовую форму: 32 шестнадцатеричные цифры с дефисами.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
data, err := uu.MarshalText()
if err != nil {
    fmt.Println(err)
}
fmt.Println(string(data))
```

Output TEXT:

```
01968727-8c7e-8000-87cb-bdba4f634d9f
```

UnmarshalBinary

Декодирует UUID из 16-байтового бинарного представления. Возвращает ошибку, если длина входных данных не равна 16 байтам.

```
var uu uuid.UUID
uuidV8Binary := []byte{0x01, 0x96, 0x87, 0x27, 0x8c, 0x7e, 0x80, 0x00, 0x87, 0x27, 0x8c, 0x7e, 0x80, 0x00, 0x87, 0x27}
err := uu.UnmarshalBinary(uuidV8Binary)
if err != nil {
    fmt.Println(err)
}
fmt.Println(uu.String())
```

Output TEXT:

```
01968727-8c7e-8000-87cb-bdba4f634d9f
```

UnmarshalJson

Декодирует UUID из Json-строки. Принимает как строки в кавычках, так и без; возвращает ошибку при неверном формате.

```
var uu uuid.UUID
uuidV8Json := []byte(`"01968727-8c7e-8000-87cb-bdba4f634d9f"`)
err := uu.UnmarshalJson(uuidV8Json)
if err != nil {
    fmt.Println(err)
}
fmt.Println(uu.String())
```

Output TEXT:

```
01968727-8c7e-8000-87cb-bdba4f634d9f
```

UnmarshalText

Декодирует UUID из канонической текстовой формы

32шестнадцатеричныецифрысдефисами. Регистронезависимый; возвращает ошибку при неверном формате.

```
var uu uuid.UUID
uuidV8Text := []byte("01968727-8c7e-8000-87cb-bdba4f634d9f")
err := uu.UnmarshalText(uuidV8Text)
if err != nil {
    fmt.Println(err)
}
fmt.Println(uu.String())
```

Output TEXT:

```
01968727-8c7e-8000-87cb-bdba4f634d9f
```

API / SQL / Методы

На этой странице описана нативная интеграция с `database/sql` для UUID. Методы `Scan` и `Value` позволяют UUID беспрепятственно работать с базами данных SQL, включая поддержку `NULL` для nullable-колонок.

NULLUUID

Scan

Реализует интерфейс `sql.Scanner` для nullable-колонок UUID. Принимает `nil` для SQL NULL или валидную строку/байты UUID. Устанавливает `Valid` в `true`, когда UUID присутствует, и в `false` для NULL.

```

var nu uuid.NullUUID
if err := nu.Scan(nil); err != nil {
    fmt.Println(err)
}
fmt.Println("Valid:", nu.Valid)
fmt.Println("UUID:", nu.UUID)
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
if err := nu.Scan(uuidV8String); err != nil {
    fmt.Println(err)
}
fmt.Println("Valid:", nu.Valid)
fmt.Println("UUID:", nu.UUID)

```

Output TEXT:

```

Valid: false
UUID: 00000000-0000-0000-0000-000000000000
Valid: true
UUID: 01968727-8c7e-8000-87cb-bdba4f634d9f

```

Value

Реализует интерфейс `driver.Valuer` для nullable-колонок UUID. Возвращает `nil`, когда `Valid` равен `false`, или строку UUID, когда `Valid` равен `true`.

```

var nu uuid.NullUUID
value, _ := nu.Value()
fmt.Println(value)
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
nu.Scan(uuidV8String)
value, _ = nu.Value()
fmt.Println(value)

```

Output TEXT:

```

<nil>
01968727-8c7e-8000-87cb-bdba4f634d9f

```

UUID

Scan

Реализует интерфейс `sql.Scanner`. Декодирует UUID из значения базы данных — принимает строку, байтовый срез или `nil` *NULL*. Возвращает ошибку, если значение не может быть разобрано.

```
var uu uuid.UUID
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
err := uu.Scan(uuidV8String)
if err != nil {
    fmt.Println(err)
}
fmt.Println(uu.String())
```

Output TEXT:

```
01968727-8c7e-8000-87cb-bdba4f634d9f
```

Value

Реализует интерфейс `driver.Valuer`. Кодировывает UUID в значение, пригодное для хранения в базе данных. Возвращает `nil` для null UUID.

```
uuidV8String := "01968727-8c7e-8000-87cb-bdba4f634d9f"
uu, err := uuid.Parse(uuidV8String)
if err != nil {
    fmt.Println(err)
}
value, err := uu.Value()
if err != nil {
    fmt.Println(err)
}
fmt.Println(value)
```

Output TEXT:

```
01968727-8c7e-8000-87cb-bdba4f634d9f
```

